

原位测试时训练

Guhao Feng^{1,2,*}, Shengjie Luo^{1,*}, Kai Hua¹, Ge Zhang¹,
Di He^{2,†}, Wenhao Huang^{1,†}, Tianle Cai¹

¹ByteDance Seed, ²Peking University

*Equal Contribution, †Corresponding authors

Abstract

静态的“先训练后部署”范式从根本上限制了大语言模型（LLMs）根据真实任务中持续流入的新信息动态调整其权重的能力。测试时训练（TTT）通过在推理时更新模型参数子集（快速权重）提供了一种有吸引力的替代方案，但其在当前的 LLM 生态系统中的潜力受到关键障碍的阻碍，包括架构不兼容、计算效率低下以及用于语言建模的快速权重目标不匹配。在这项工作中，我们引入了 原位测试时训练（*In-Place TTT*），这是一种能够无缝赋予 LLM 测试时训练能力的框架。*In-Place TTT* 将普遍存在的 MLP 块的最终投影矩阵视为其可调整的快速权重，从而实现对 LLM 的“即插即用”式增强，无需昂贵的从头再训练。此外，我们用一种量身定制、有理论依据且明确与自回归语言建模的“下一个 Token 预测”任务对齐的目标，替换了 TTT 的通用重构目标。这一原则性目标与高效的分块更新机制相结合，产生了一种高度可扩展且兼容上下文并行的算法。大量实验验证了我们框架的有效性：作为原位增强，它使一个 4B 参数的模型能够在上下文长度达 128k 的任务上取得优异表现；而当从头开始预训练时，它持续优于具有竞争力的 TTT 相关方法。消融实验结果进一步为我们设计选择提供了更深入的见解。总的来说，我们的结果确立了 *In-Place TTT* 是迈向 LLM 持续学习范式的一个有希望的步骤。

Date: 2026 年 5 月 14 日

Correspondence: Di He at dihe@pku.edu.cn and Wenhao Huang at huang.wenhao@bytedance.com

Code: <https://github.com/ByteDance-Seed/In-Place-TTT>

1 引言

大型语言模型 (LLMs) 在一系列复杂任务中展现了显著的能力 [8, 11, 39, 51]。这一成功主要建立在静态的“先训练后部署”范式之上，即模型首先从大规模语料库中获取知识，然后在推理过程中保持固定。然而，这种设计带来了一个根本性限制：一旦部署，模型的权重就无法更新，从而无法动态适应流式输入 token 提供的特定上下文。因此，在测试时，模型在处理和推理长期、不断演变的任务 [9, 45] 以及像人类一样从无限的经验流中持续学习 [44] 的能力上受到限制。

情境学习 [8, 56] 通过在上下文中保留所有过去的输入 token，提供了一种缓解此问题的方法。然而，其有效性受限于模型的上下文窗口，而上下文窗口又受到事实上的注意力机制二次复杂度 [52] 的限制。这一瓶颈推动了旨在高效扩展上下文窗口的架构解决方案的研究方向 [6, 10, 17, 42]。不同的是，测试时训练 (TTT) 已成为一种新范式 [3, 47, 48, 53, 63]。TTT 不仅仅使静态模型更高效，还使模型能够动态更新参数并适应任何特定上下文，直接针对前述限制。具体来说，TTT 引入了一小部分模型参数，称为快速权重 [43]，这些参数可以为每个新输入即时更新。通过最小化自监督重构目标，这些快速权重压缩并内化上下文信息，充当一种表达性的在线演化状态。

尽管其概念上具有吸引力，但在当前 LLM 生态系统中释放 TTT 的潜力受到关键障碍的阻碍：(i) 现有的 TTT 方法通常依赖于标准 Transformer 块之外的专业化层，这通常需要昂贵的从头预训练才能获得令人满意的性能 [47, 48, 53, 67]；(ii) 典型的 TTT 机制本质上是顺序的 [47, 48]。虽然现有工作探索了逐块加速 [3, 30, 49, 63]，但 TTT 作为主要 token 混合器的角色迫使其依赖小块来维持性能，从而成为充分利用现代加速器所需的大规模并行化的瓶颈；以及 (iii) 普遍使用通用重构目标来更新 TTT 的快速权重，并未针对主导自回归语言模型的因果性下一 token 预测任务进行显式定制，可能阻碍其最终性能。

为了弥合这一差距，我们引入 原位测试时训练 (*In-Place TTT*)，这是一个通过直接解决上述障碍而设计用于无缝赋予 LLMs 测试时训练能力的框架。我们的核心见解是利用原位设计重新利用现有的 MLP 块，而不是引入新的专业化层（[应对障碍 i](#)）。具体来说，In-Place TTT 将 MLP 块的最终投影矩阵视为其快速权重，并在推理过程中原位更新它。这种“即插即用”设计无需修改模型架构，保持预训练权重的完整性，并实现即时适应，无需昂贵的从头重新训练。

为了解决计算效率低下和目标错位的问题，我们进一步设计了一种针对语言建模的定制化适应机制。遵循先前的工作 [3, 30, 49, 63, 67]，我们用可扩展的逐块更新规则替代低效的逐 token 更新（[应对障碍 ii](#)）。此外，我们的原位设计与注意力机制互补运行。这种协同作用消除了独立 TTT 层所需的小块需求，从而确保在现代加速器上的高吞吐量。同时，我们超越了先前工作的通用重构目标 [48, 67]，并引入一种与下一 token 预测 (NTP) 目标显式对齐的新型目标（[应对障碍 iii](#)）。基于严格的理论分析，我们表明这种与 NTP 对齐的目标鼓励快速权重存储对自回归语言建模具有预测有用性的信息，从而产生一种高效且可扩展的算法。

基于这些原则性的设计选择，我们的 In-Place TTT 提供了一个实用且有效的框架，用于通过动态、持续的自适应增强 LLMs。我们在不同计算规模的语言建模任务上进行了大量实验，将其作为实用代理来探索模型在长期、不断演变的任务上的潜力。通过相对廉价的持续训练，我们的 In-Place TTT 使 Qwen3-4B-Base 在上下文长度达 128k 的任务上实现卓越性能。此外，我们将 In-Place TTT 与有竞争力的 TTT 相关方法进行比较，通过在长度达 32k 的语料库上进行从头预训练，验证了我们框架的架构优势。最后，关于状态大小、块大小和快速权重目标的消融研究提供了更深入的见解，证实了每个设计选择的关键作用。总的来说，我们的结果将 In-Place TTT 确立为迈向 LLMs 持续学习范式的有希望的一步。

2 预备知识：测试时训练

本节介绍测试时训练(TTT)，这是一种使模型能够在推理时动态适应新数据的范式 [47, 48, 67]。我们首先详细阐述 TTT 机制，然后讨论成功将 TTT 应用于 LLM 的关键需求，这直接激发了我们的框架。

TTT 机制。其核心在于，TTT 机制利用快速权重 [2, 43]，记为 $\mathbf{W}$ 。这些权重构成一个小型神经网络 $f_{\mathbf{W}}(\cdot) : \mathbb{R}^d \rightarrow \mathbb{R}^d$ ，该网络在测试时快速更新。与训练后冻结的标准模型权重不同，快速权重 $\mathbf{W}$ 充当动态内存，持续存储和检索序列中的上下文信息。

为了处理输入序列 $\mathbf{x} = [x_1, x_2, \dots, x_N]$ ，每个 token $x_i \in \mathbb{R}^d$ 通常会被投影以推导出 TTT 操作所需的输入，例如查询 (q_i)、键 (k_i) 和值 (v_i)。然后，TTT 机制通过两个核心的顺序操作运行：

1. **更新操作：**快速权重 $\mathbf{W}$ 被更新，以将键 k_i 与其对应的值 v_i 关联起来。这被构建为一个单一的优化步骤，最小化损失函数 $\mathcal{L}(\cdot, \cdot)$ （例如，均方误差），该函数衡量这种关联中的差异。直观上，该步骤将 (k_i, v_i) 对中的信息编码到神经记忆 $f_{\mathbf{W}}$ 中。给定学习率 η ，更新规则为：

$$\mathbf{W}_i \leftarrow \mathbf{W}_{i-1} - \eta \nabla_{\mathbf{W}} \mathcal{L}(f_{\mathbf{W}_{i-1}}(k_i), v_i).$$

2. **应用操作：**新更新的网络 $f_{\mathbf{W}}$ ，现在由 $\mathbf{W}_i$ 参数化，用于处理查询 q_i ，即 $o_i = f_{\mathbf{W}_i}(q_i)$ 。该输出 o_i 融入了来自先前键值对的上下文信息，因为这些信息现在被编码在 $\mathbf{W}_i$ 中。

尽管这一两步公式描述了 TTT 的高层机制，但具体的实现细节可能差异很大。实际上，最近许多研究探索了丰富的设计空间，包括不同的损失函数、更复杂的优化器以及替代的神经记忆参数化方式，以提高性能和效率 [3, 5, 32, 54]。这些设计选择关键地影响着快速权重存储、检索和遗忘序列信息的效率，从而使 TTT 机制适用于不同的数据模态和任务。

TTT 在 LLM 生态系统中的关键需求。尽管 TTT 作为动态适应范式具有前景，但在 LLM 生态系统中释放其潜力需要应对几个关键挑战。要使 TTT 成为一个可行且有效的组件，它必须满足以下需求：

- **架构兼容性。**如果一种架构能够从预训练检查点热启动，我们称其与 LLM 兼容。然而，当前的 TTT 机制通常被开发为独立的循环层，旨在替代注意力而非补充注意力 [29, 47, 48, 53, 67]。这需要昂贵的从头预训练，为主宰 LLM 生态系统的大规模、数十亿参数模型带来了显著的采用障碍。因此，一个关键的需求是设计一种不需要根本性架构修改的“即插即用”设计。
- **计算效率。**该机制必须在现代并行加速器上高效运行。TTT 的经典逐 token 更新规则本质上是顺序的，因此严重限制了 GPU 和 TPU 的并行处理能力 [3, 47, 48]。这种操作的低效率使得细粒度更新对于高吞吐量语言建模变得不切实际。因此，高效的 TTT 实现必须超越逐 token 方案并确保可扩展性，例如采用分块更新机制 [3, 30, 36, 49]。
- **针对语言建模的定制学习目标。**TTT 中主要的自监督目标是重构，其中模型学习关联 (k_i, v_i) 对，且 v_i 通常来源于输入 token x_i 本身 [29, 47, 48, 53, 67]。虽然这个通用目标使 TTT 机制能够存储信息，但它与语言建模的最终目标——预测下一个 token——的直接相关性并无保证。目标值 v 的选择仍然是一个关键但尚未充分探索的设计决策，可能对于捕捉 LLM 所需的复杂因果依赖关系而言并非最优。

3 原地测试时训练

为了满足section 2中概述的需求，我们引入了**原地 测试时训练 (In-Place TTT)**，这是一个旨在解锁 LLMs 的 TTT 能力的框架。我们首先介绍我们的整体框架，该框架通过一种原地设计（重用现有的 MLP 块）解决了架构不兼容问题，同时通过分块更新机制确保计算效率（section 3.1）。然后我们详细介绍我们新颖的 LM 对齐目标，该目标通过对齐下一个 Token 预测（NTP）目标而为 LLMs 明确设计（section 3.2）。随后，我们对目标的优越性质进行了理论分析（section 3.3），并以实际实现细节作为结束（section 3.4）。

3.1 总体框架

将 MLP 块改造为原位适配。先前的 TTT 研究大多将其定位为替代注意力机制的潜在方案。然而，这些先前研究通常在中等规模下进行，这与现代数十亿参数的大语言模型完全不同。因此，替换核心注意力机制（其习得属性对大语言模型能力至关重要）是一种高风险架构修改。此外，引入任何新的随机初始化层也会与 LLM 数十亿训练参数产生冲突，需要昂贵且通常不可行的重新训练来解决这种不平衡。

我们的核心思路是完全绕开这些挑战。我们不替换或添加组件，而是改造一个通用模块——多层感知机（MLP）块——使其同时充当快速权重。回顾 section 2 中的 TTT 公式，快速权重的选择没有限制，即任何参数都可以通过 TTT 机制更新作为快速权重。特别地，Transformer 中的 MLP 块也可以被视为一种键值记忆 [22]，作为预训练期间获取的广泛通用知识的“慢速权重”。因此，利用同一组件同时作为自适应的“快速权重”，在推理时动态内化瞬时的上下文信息，是一种自然的扩展。

形式上，我们适配广泛使用的门控 MLP 架构 [24, 57]。给定隐藏表示 $\mathbf{H}$ ，门控 MLP 计算其输出表示 $\mathbf{O} = (\phi(\mathbf{H}\mathbf{W}_{\text{gate}}^\top) \odot (\mathbf{H}\mathbf{W}_{\text{up}}^\top)) \mathbf{W}_{\text{down}}^\top$ 。在我们的框架中，我们将输入投影 $\mathbf{W}_{\text{up}}$ 和 $\mathbf{W}_{\text{gate}}$ 视为冻结的慢速权重，而将最终投影矩阵 $\mathbf{W}_{\text{down}}$ 改造为可适配的快速权重。通过仅原地更新 $\mathbf{W}_{\text{down}}$ ，我们保留了模型的架构完整性，将 TTT 从破坏性重构转变为 LLM 的轻量级“即插即用”增强。

基于分块更新的高效适配。除了架构兼容性，我们的原位设计还带来了显著的计算效率提升。传统的 TTT 方法由于旨在替代注意力机制，被迫采用低效的逐 token 更新，以强制执行严格因果性并进行细粒度的 token 混合。近期研究探索了分块更新方法以加速 [3, 30, 36, 49]。我们的框架同样遵循这一思路，完全绕开了权衡。由于我们仅适配 MLP 块，而保持注意力层不变，因此摆脱了逐 token 约束，实现了更高效的分块更新策略，进一步避开了小分块限制（我们的消融研究结果（第 4.3 节）也验证了我们的框架天然适合分块更新——特别是大分块更新——在分块大小为 512 到 1024 时达到最优性能）。

该过程如下进行。给定中间激活 $\mathbf{Z} = \phi(\mathbf{H}\mathbf{W}_{\text{gate}}^\top) \odot (\mathbf{H}\mathbf{W}_{\text{up}}^\top) \in \mathbb{R}^{n \times d_{\text{ff}}}$ 以及对应的值目标和输出 $\mathbf{V}, \mathbf{O} \in \mathbb{R}^{n \times d_{\text{model}}}$ ，我们将它们划分为 k 个大小为 C 的无重叠分块，记为 $\square_{[i]} = \square_{iC+1:(i+1)C} \in \mathbb{R}^{C \times d'}$ ，其中 $\square \in \{\mathbf{Z}, \mathbf{V}, \mathbf{O}\}, i \in [k]$ ， d' 为对应维度。设 $\mathbf{W}_{\text{down}}^{(i)}$ 为处理分块 i 之前的快速权重状态，且 $\mathbf{W}_{\text{down}}^{(0)} = \mathbf{W}_{\text{down}}$ 。对于每个分块 $i \in [k]$ ，我们执行两个顺序操作：

1. **应用操作：**快速权重的当前状态 $\mathbf{W}_{\text{down}}^{(i)}$ 用于处理块 $\mathbf{Z}_{[i]}$ ，即 $\mathbf{O}_{[i]} = \mathbf{Z}_{[i]}(\mathbf{W}_{\text{down}}^{(i)})^\top$ 。
2. **更新操作：**快速权重 $\mathbf{W}_{\text{down}}^{(i)}$ 使用 $\mathbf{Z}_{[i]}$ 作为键、 $\mathbf{V}_{[i]}$ 作为值进行更新，通过一个梯度下降步骤执行，损失函数为 $\mathcal{L}$ ，学习率为 η ：
$$\mathbf{W}_{\text{down}}^{(i+1)} = \mathbf{W}_{\text{down}}^{(i)} - \eta \nabla_{\mathbf{W}} \mathcal{L} \left(\mathbf{Z}_{[i]}(\mathbf{W}_{\text{down}}^{(i)})^\top, \mathbf{V}_{[i]} \right)。$$

这种分块更新策略针对现代硬件设计，类似先前工作 [3, 30, 36, 49]。此外，由于我们的原位 MLP 适配，我们可以使用大分块大小 C 一次性处理大量 token，从而高度利用并行性并充分利用 GPU 或 TPU 的计算能力。

3.2 语言模型对齐目标

在建立了高效的就地适应框架后，In-Place TTT 的性能现在取决于其学习目标的设计。在本节中，我们介绍我们的语言模型对齐（Language Modeling-Aligned）目标，该目标专门为 LLMs 定制。

先前的 TTT 方法通常使用重建目标，例如 $\mathcal{L}(f_{\mathbf{w}}(k), v)$ ，其中 k 和 v 都是同一输入 token x 的线性投影输出 [47, 48, 67]，这鼓励模型仅仅记忆当前 token 的表示。我们认为这对于语言建模任务来说是次优的。相反，我们建议将目标与支配 LLMs 的下一个 token 预测（Next-Token Prediction, NTP）目标对齐。

为了实现这一点，我们将目标 v 指定为包含未来 token 信息。形式上，我们推导出目标 $\hat{\mathbf{V}} = \text{Conv1D}(\mathbf{X}_0)\mathbf{W}_{\text{target}}$ ，其中 $\mathbf{X}_0 \in \mathbb{R}^{n \times d_{\text{model}}}$ 表示 token 嵌入， $\text{Conv1D}(\cdot)$ 是一维卷积算子， $\mathbf{W}_{\text{target}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ 是一个可训练的投影矩阵。在此公式下，未来 token 信息的数量可以在我们的目标 $\hat{\mathbf{V}}$ 中控制，例如，通过将 $\mathbf{W}_{\text{target}}$ 参数化为恒等变换，并分配 $\text{Conv1D}(\cdot)$ 的核权重使得下一个 token 为 1，其他 token 为 0，可以实现下一个 token 的目标。

通过这个对齐的目标，我们为了简单起见，使用广泛使用的相似度度量来实例化我们的损失函数，即 $\mathcal{L}(\cdot, \cdot) = -\langle \cdot, \cdot \rangle_F$ 。在此损失函数下，关于我们分块机制中快速权重的梯度可以直接推导出：

$$\mathbf{W}_{\text{down}}^{(i)} = \mathbf{W}_{\text{down}}^{(i-1)} + \eta \hat{\mathbf{V}}_{[i]}^\top \mathbf{Z}_{[i]}. \quad (1)$$

3.3 理论分析

直观上，我们的 LM-Aligned 目标明确鼓励快速权重压缩对未来 token 具有预测价值的信息，从而增强模型的动态适应能力。在本小节中，我们通过理论分析我们目标的好处来形式化这一直觉。我们将分析扎根于经典的归纳头设定 [19, 38]，这是一种被认为是 LLMs 中情境学习关键机制的机制。

设置。 考虑一个输入序列，其中键值对 $(x_{t^*}, x_{t^*+1}) = (k^*, v^*)$ 出现在任意位置 t^* 。随后，在查询位置 $n > t^*$ ，键 k^* 再次出现，使得 $x_n = k^*$ 。模型必须正确预测关联的值 $x_{n+1} = v^*$ 。

不失一般性，我们分析一个由我们的 In-Place TTT 增强的单一 Transformer 块。令 $\mathbf{Z}_t \in \mathbb{R}^{d_{\text{ff}}}$ 为 token x_t 的中间激活， $\mathbf{E}_w \in \mathbb{R}^{d_{\text{model}}}$ 为 x_w 的 token 嵌入。在我们的框架中，来自先前上下文块的快速权重更新为 $\Delta \mathbf{W}_{\text{down}} = \eta \sum_{t \in \text{prior chunks}} \mathbf{V}_t \mathbf{Z}_t^\top$ 。此更新随后改变查询位置 n 处的输出 logit 为 $\Delta \ell_n[w] = \mathbf{E}_w^\top \Delta \mathbf{W}_{\text{down}} \mathbf{Z}_n$ 。我们比较 TTT 目标 $\mathbf{V}_t$ 的两种选择：

- 重建目标: $\mathbf{V}_t^{\text{rec}} = \mathbf{E}_{x_t}$ ，当前 token 的嵌入。
- LM 对齐目标: $\mathbf{V}_t^{\text{LM}} = \mathbf{E}_{x_{t+1}}$ ，下一个 token 的嵌入。

假设。 我们的分析依赖于关于嵌入和中间激活特性的两个温和假设，这些假设在 Transformer 的理论分析中是标准的：

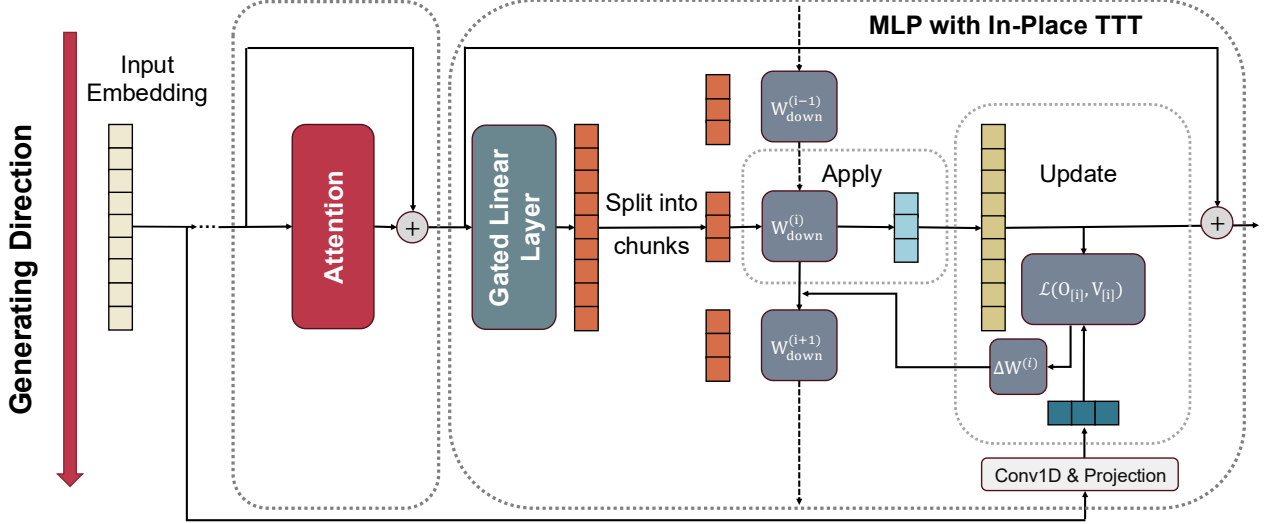


Figure 1 我们的原位测试时训练的整体框架。该模块按顺序对输入块进行操作。对于每个块，首先将当前快速权重 应用到中间激活值 $\mathbf{Z}$ 以产生输出。然后，使用激活值 $\mathbf{Z}$ 和从 token 嵌入导出的值 $\mathbf{V}$ 来更新这些权重。这种“先应用后更新”的循环使得模型能够以严格因果的方式动态适应传入的上下文。

1. 嵌入的近似正交性：对于任意两个不同的 token $w_i, w_j \in \mathcal{V}$ ，它们的嵌入近似正交： $|\mathbf{e}_{w_i}^\top \mathbf{e}_{w_j}| \leq \epsilon$ ，其中 $\epsilon > 0$ 是一个小常数。此外，嵌入具有非平凡的大小： $\|\mathbf{e}_{w_i}\|^2 \geq c_{\text{norm}}^2 > 0$ ，其中 c_{norm} 为某个常数。
2. 键-查询对齐：查询 token $x_n = k^*$ 的中间激活值 $\mathbf{Z}_n$ 与其对应键 token $x_{t^*} = k^*$ 的 $\mathbf{Z}_{t^*}$ 对齐： $\mathbb{E}[\mathbf{z}_{t^*}^\top \mathbf{z}_n] = c_{\text{align}} > 0$ 。对于其他位置 $t \neq t^*$ ，这些 token 与查询无关，即 $\mathbb{E}[(\mathbf{V}_t \mathbf{Z}_t^\top) \mathbf{Z}_n] = \mathbf{0}$ 。

有了这一设定，我们呈现我们的主要理论结果：

Theorem 1 (LM 对齐目标与重建目标的 Logits-wise 效果). 在指定设置和假设下，对于学习率 $\lambda_{lr} > 0$ ，使用 LM 对齐目标 进行一次更新步骤后，logits 的期望变化 $\Delta \ell_n$ 满足：

$$(\text{正确 logits 增加}) \quad \mathbb{E}[\Delta \ell_n[v^*]] \geq \lambda_{lr} \cdot c_{\text{norm}}^2 \cdot c_{\text{align}}, \quad (2)$$

$$(\text{其他 logits 几乎不变}) \quad |\mathbb{E}[\Delta \ell_n[w]]| \leq \lambda_{lr} \cdot \epsilon \cdot c_{\text{align}}, \quad \forall w \neq v^*. \quad (3)$$

相比之下，对于重建目标，正确 token 的 logits 期望变化可以忽略不计： $|\mathbb{E}[\Delta \ell_n[v^*]]| \leq \lambda_{lr} \cdot \epsilon \cdot c_{\text{align}}$ 。

证明在附录 A 中提供。在 theorem 1 中，LM-Aligned 目标在期望上保证增加正确下一个 token v^* 的 logit，并保持其他 token 的 logit 近似不变，直接辅助模型的预测任务。相比之下，重建目标不提供这样的预测益处，未能增加正确 token 的 logit。在实践中，我们的实现将此原则从单一下一个 token 扩展到学习到的局部未来 token 组合，这也与高级 LLMs 中 Multi-Token Prediction 的最新有前景结果 [37] 相一致，作为 NTP 目标的有效扩展。这使得我们的 In-Place TTT 能够捕获更丰富的预测信号，从而比简单重建更有效地压缩有用的上下文信息。

3.4 实现细节

结合前述设计，figure 1 展示了我们的 In-Place TTT 框架。这里我们进一步阐述实际实现细节，这些细节旨在现代硬件上实现高效性和可扩展性。特别是，我们的方法与上下文并行 (CP) 完全兼容，依赖于并行扫描算法同时处理序列块，同时保持自回归更新的严格因果语义。此外还提供了进一步的讨论。

利用上下文并行的高效实现。 我们在 equation (1) 中的更新规则具有结合性，使得 In-Place TTT 适用于上下文并行实现，该实现沿序列长度进行划分并同时处理各个块。该过程分为三个阶段：(i) 对于所有块 $i \in \{1, \dots, T\}$ ，我们并行计算中间激活 $\mathbf{Z}_{[i]}$ 和快速权重更新 $\Delta \mathbf{W}_{\text{down}}^{(i)} = (\hat{\mathbf{V}}_{[i]})^\top \mathbf{Z}_{[i]}$ ；(ii) 对 $[\dots, \Delta \mathbf{W}_{\text{down}}^{(i)}, \Delta \mathbf{W}_{\text{down}}^{(i+1)}, \dots]$ 执行一次前缀和，以计算每个块的聚合更新： $\Delta \mathbf{S}_i = \sum_{j=1}^{i-1} \Delta \mathbf{W}_j$ ，这在现代加速器上可以非常高效；(iii) 每个块的有效快速权重 $\mathbf{W}_{\text{down}}^{(i-1)} = \mathbf{W}_{\text{down}}^{(0)} + \eta \Delta \mathbf{S}_i$ 以及相应输出 $\mathbf{O}_{[i]} = \mathbf{Z}_{[i]}(\mathbf{W}_{\text{down}}^{(i-1)})^\top$ 被并行计算。

因果性与边界处理。 为了确保块 i 的更新 delta 本身不包含未来信息，我们在生成 value 时对一维卷积应用因果填充。这使每个 delta 计算局限于其对应的块，从而使并行扫描在数学上等价于顺序更新。此外，在文档边界处，快速权重被重置为预训练状态，以防止上下文在独立序列之间泄漏。最终的上下文并行算法见附录 B 中的 algorithm 1。

讨论。 总之，我们的 In-Place TTT 实现协同地结合了简单、计算高效的更新规则和并行扫描算法。这种设计选择使我们的方法不仅快速且可扩展，而且在数学上等价于严格因果的顺序过程，这得益于精心的边界和填充管理。得到的模块是 CP 原生的，完全因果，并且可以作为标准 Transformer 架构中 MLP 块的即插即用替代品无缝集成。最后，值得注意的是，我们框架的核心原则与特定的损失函数及其优化器选择是正交的，这一点已在更广泛的 TTT 文献中得到广泛研究，我们将其作为未来工作的一个有前景方向留待探索。

4 实验

在本节中，我们进行了一系列实验，以实证验证我们提出的 In-Place TTT 框架的有效性。具体来说，我们旨在回答以下研究问题：

- **Q1:** In-Place TTT 能以多高的效率以“即插即用”方式增强预训练 LLM？
- **Q2:** 从头开始训练时，In-Place TTT 与此前的 TTT 方法相比效果如何？
- **Q3:** In-Place TTT 框架中关键设计选择的影响是什么？

我们将不同规模的语言建模任务作为实际代理，在以下小节中通过精心设计的实验来回答每个问题。由于篇幅限制，我们在附录 C 中详细描述了实验设置。

4.1 In-Place TTT 作为预训练 LLMs 的即插即用增强

为了验证 In-Place TTT 作为现有预训练 LLMs 的“即插即用”增强，我们从具有竞争力的开源模型 *Qwen3-4B-Base* 开始。其原始上下文窗口为 32k，因此我们可以通过不同上下文长度的语言建模任务，模拟需要测试时训练 (Test-Time Training) 能力的长期演进任务。具体而言，我们比较以下两种模型

Table 1 关于 RULER [28] 的评估结果。报告了平均准确率 (%), 最佳结果以 **粗体** 显示。

Model	In-Domain Evaluation						Extrapolation
	4k	8k	16k	32k	64k	128k	256k
Mistral-7B [31]	93.6	91.2	87.2	75.4	49.0	13.8	-
GLM3-6B [23]	87.8	83.4	78.6	69.9	56.0	42.0	-
Phi3-medium-14B [1]	93.3	93.2	91.1	86.8	78.6	46.1	-
Llama3-8B [41]	92.8	90.3	85.7	79.9	76.3	69.5	-
Qwen3-4B (Instruct) [57]	95.1	93.6	91.0	87.8	77.8	66.0	-
Baseline	96.6	94.1	92.1	88.7	74.3	74.8	41.7
In-Place TTT	96.1	95.6	92.7	89.3	78.7	77.0	43.9

的性能: (1) *Qwen3-4B-Base* (基线); (2) *Qwen3-4B-Base + In-Place TTT* (In-Place TTT)。两个模型经历完全相同的持续训练课程, 确保公平比较, 其中我们的 In-Place TTT 是唯一的变量。

训练与评估。 持续训练课程分为两个阶段: 初始阶段约 ~20B tokens, 上下文长度为 32k; 第二阶段约 ~15B tokens, 上下文长度为 128k。训练数据集的详细描述见附录 C.1。为有效管理这些长序列, 我们使用 YaRN [42] 适配模型的旋转位置嵌入 (Rotary Position Embeddings)。我们在 RULER 基准 [28] 上评估两个模型的长上下文性能, 使用流行的 OpenCompass 框架 [13], 上下文长度范围从 4k 到 256k。其中 256k 设置专门测量模型外推至 128k 上下文长度限制之外的能力。训练细节的详细描述见附录 C.2。

结果与讨论。 结果总结于 table 1, 表明 In-Place TTT 显著提升了预训练模型的长上下文能力。具体而言, 从结果中可以清晰看出一个明显趋势: 虽然两个模型在短上下文中具有竞争力, 但经过我们的 In-Place TTT 增强的 Qwen3-4B-Base 随着序列长度增加, 建立了持续且不断扩大的优势。它在 64k 和 128k 上下文长度上取得了显著提升。关键的是, 当外推至 256k 上下文时, 这一优势得以保持, 展现出卓越的泛化能力。这些发现证实, In-Place TTT 可以无缝集成到预训练 LLM 中, 以提升其长上下文能力。模型在上下文长度内及超出时的强劲表现验证了我们的方法作为扩展现有 LLM 能力的实用且强大的工具。

扩展到更多模型。 为验证我们方法的通用性, 我们将 In-Place TTT 作为即插即用增强应用于另外两个模型: LLaMA-3.1-8B [24] 和 Qwen3-14B-Base [57], 遵循相同的持续训练协议。如表 2 所示, In-Place TTT 在两个模型的所有上下文长度上一致地提升了 RULER 分数。提升在较长上下文上尤为显著, 例如 LLaMA-3.1-8B 在 64k 时提升 +2.1, Qwen3-14B-Base 在 64k 时提升 +2.7。此外, 当与用于位置外推的 YaRN [42] 结合时, 改进仍然保持, 证实我们的方法与 RoPE 扩展技术是正交的。这些结果跨越不同的模型系列和规模 (4B-14B), 进一步强化了 In-Place TTT 是一种广泛适用的预训练 LLM 即插即用增强方法。

4.2 从头预训练: 对比分析

在展示了 In-Place TTT 作为“即插即用”模块的有效性之后, 我们进一步评估其集成到从头预训练模型中的性能和可扩展性。我们的分析分两个阶段: 首先在 500M 和 1.5B 规模上建立其语言建模能力,

Table 2 将 In-Place TTT 扩展到 LLaMA-3.1-8B 和 Qwen3-14B-Base 在 RULER 基准测试上的表现。我们报告平均准确率 (%), 最佳结果以 **粗体** 显示。

Base Model	Method	4k	8k	16k	32k	64k	64k+YaRN
LLaMA-3.1-8B	Baseline	93.9	92.1	92.5	91.1	81.6	–
	In-Place TTT	94.4	93.0	93.3	91.7	83.7	–
Qwen3-14B	Baseline	96.8	95.0	94.6	90.7	67.9	81.3
	In-Place TTT	97.2	95.7	95.2	91.2	70.6	82.5

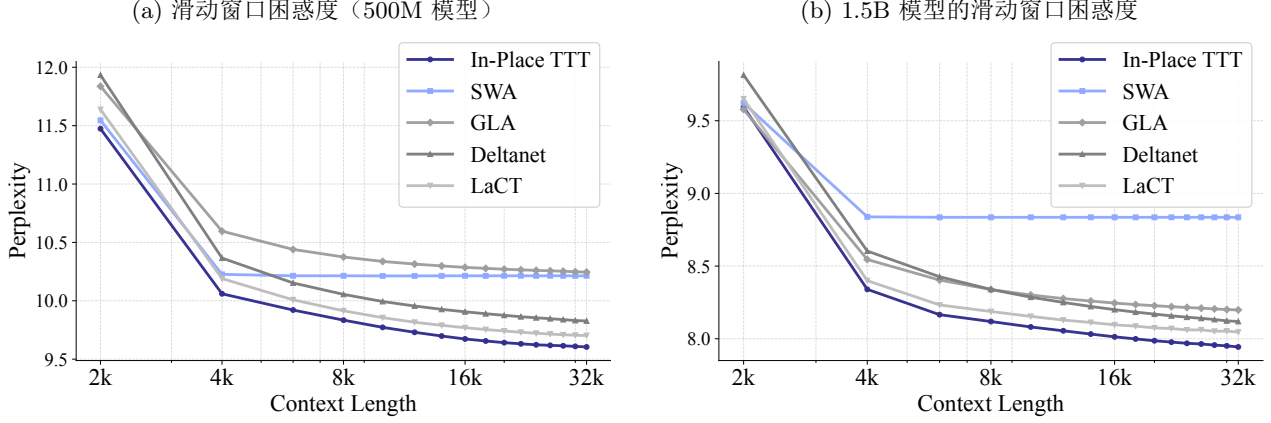


Figure 2 在不同上下文长度下, 在 Pile 数据集上, 针对 500M (左) 和 1.5B (右) 参数模型的滑动窗口困惑度。我们的 In-Place TTT 一致地实现了比所有竞争基线更低的困惑度。

		Common Sense Reasoning					Long-Context Evaluation		
Model Architecture		HellaSwag	ARC-E	ARC-C	MMLU	PIQA	RULER-4k	RULER-8k	RULER-16k
Baselines	Full Attn.	55.67	64.52	33.19	36.43	72.63	45.77	38.09	6.58
	SWA	54.92	64.18	32.85	36.06	72.58	14.77	9.91	5.07
I.P. TTT	Full Attn.	55.85	64.98	32.34	37.42	73.29	49.98	43.82	19.99
	SWA	55.24	64.60	33.70	36.48	72.03	28.33	26.80	7.57

Table 3 4B 模型在常识推理和长上下文评估基准上的评估结果。最佳性能以 **粗体** 显示。“SWA”即滑动窗口注意力 (Sliding-Window Attention), “Full Attn.”即完全注意力 (Full Attention), “I.P. TTT”即我们的原地 TTT (In-Place TTT)。

然后评估其在更大 4B 模型上的可扩展性和影响。

实验设置. 首先, 我们在 500M 和 1.5B 参数规模上, 将我们的 In-Place TTT 与先前的 TTT 相关方法和基于 TogetherAI [50] 的高效注意力方法进行基准测试。比较了多种竞争基线: (1) 带有滑动窗口注意力 (SWA) 的标准 Transformer [6, 10]; (2) 门控线性注意力 (GLA) [60]; (3) DeltaNet [43, 59, 62]; (4) 大块测试时训练 (LaCT) [67]。为公平比较, In-Place TTT 和 LaCT 均构建在 SWA 主干网络上。所有模型均在 32k 上下文长度的序列上训练。

基于这些结果, 我们进一步扩展到 4B 参数模型, 以评估 In-Place TTT 方法的可扩展性。具体来说, 我们将具有全注意力的 Transformer 和具有 SWA 的 Transformer 与经过 In-Place TTT 增强的对应模

型进行比较。这些模型在 8k 上下文长度下训练了 120B 个 token。数据集、模型配置和训练程序的详细描述见 [sections C.1 to C.3](#)。

评估. 对于 500M 和 1.5B 模型, 我们使用 滑动窗口困惑度 在由 Pile [20] 和 Proof-Pile-2 [40] 组成的验证集上评估其长上下文利用能力。该指标测量在扩展前文上下文时固定最终 token 块上的困惑度, 其中困惑度下降趋势表示有效的上下文利用。对于 4B 模型, 我们在下游任务套件上进行更广泛的评估, 包括常识推理基准 (HellaSwag [66]、ARC [12]、MMLU [26, 27]、PIQA [7]) 和长上下文 RULER 基准 [28]。

结果与讨论. 在图 2 中, 我们绘制了 500M/1.5B 模型的滑动窗口困惑度随上下文长度的变化。可以清楚地看到, 我们的 In-Place TTT 一致地获得了比所有竞争基线更低的验证困惑度, 其性能在达到完整的 32k 上下文时持续改善。这种持续改进表明其核心机制成功压缩并利用了传入上下文的信息。

此外, 表 3 中的结果进一步表明, 具有全注意力和 SWA 的 4B 参数 Transformer 在大多数常识推理任务上均得到一致改进。此外, 采用 In-Place TTT 的模型在长上下文评估上取得了更优性能, 例如, 对于具有全注意力的 Transformer, RULER-16k 分数从 6.58 提高到 19.99; 对于 SWA 模型, RULER-8k 分数从 9.91 提高到 26.80。这些显著的提升, 尤其是在各种规模的模型上, 确立了 In-Place TTT 作为一种高效且可扩展的方法。

4.3 消融实验: 关键设计选择的影响

最后, 我们使用一个 1.7B 参数的模型对 RULER 进行了一系列消融实验, 为我们的设计选择提供了更深入的见解。详细设置见附录 C.3 和 C.2。

状态大小的影响. 我们首先研究了性能如何随快速权重大小扩展, 这可以通过改变启用 TTT 的层数来控制。图 3 (a) 显示了一个清晰的趋势: 我们的 In-Place TTT 的性能随着状态大小的扩展而持续提升。这证实了更大的快速权重使模型能够更有效地适应上下文信息, 进一步支持了我们利用大量 MLP 状态的复用方法。

块大小的影响. 在 [section 3.1](#) 中, 块大小 C 控制着快速权重更新的粒度和并行度, 揭示了效率与性能之间的权衡。通过改变块大小, 图 3 (b) 显示 $C = 512$ 和 $C = 1024$ 相比其他选择都竞争性地取得了更好的性能, 而 $C = 1024$ 具有更好的效率。

LM 对齐目标的影响. 接下来, 我们深入研究在 [section 3.2](#) 中自定义的 LM 对齐目标, 其中目标定义为 $\hat{\mathbf{V}} = \text{Conv1D}(\mathbf{X}_0)\mathbf{W}_{\text{target}}$ 。具体来说, Conv1D 算子用于生成包含未来 token 信息的目标, 而 $\mathbf{W}_{\text{target}}$ 是一个投影变换。在图 3 (c) 中, 我们全面消融了这些组件的组合。结果表明, 两者对于性能保证都是必要的, 其中 Conv1D 在长上下文中起关键作用, 而 $\mathbf{W}_{\text{target}}$ 在短上下文中至关重要。这些结果与我们在 [section 3.3](#) 中的理论分析一致, 有力地支持了我们为语言建模推导自定义目标的动机。

In-Place TTT 的效率影响. 我们进一步研究了 In-Place TTT 引入的计算开销。在 [figure 4](#) 中, 我们比较了使用和不使用 In-Place TTT 时的预填充吞吐量和内存消耗。结果确实验证了我们实际实现的效率。

5 相关工作

测试时训练 (TTT). 测试时训练 (TTT) 是一种范式, 它通过更新模型的一小部分参数 (称为快速权重 [2]), 使模型在推理时能够动态适应持续的数据流。该范式最初在计算机视觉领域取得成功 [47, 53], 随

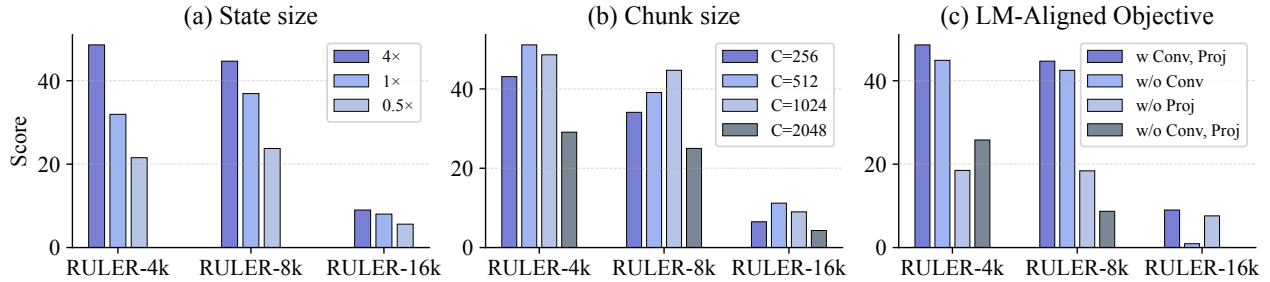


Figure 3 在 In-Place TTT 框架的关键设计选择上的消融研究，使用 1.7B 参数模型在 RULER 基准上进行评估。这些图展示了以下因素的影响：(a) 状态大小，表明性能随状态大小扩展而提升；(b) 块大小，展示了性能权衡，中间大小（例如 512、1024）为最优；(c) LM-Aligned Value 目标，确认了卷积（w Conv）和投影（w Proj）都至关重要。

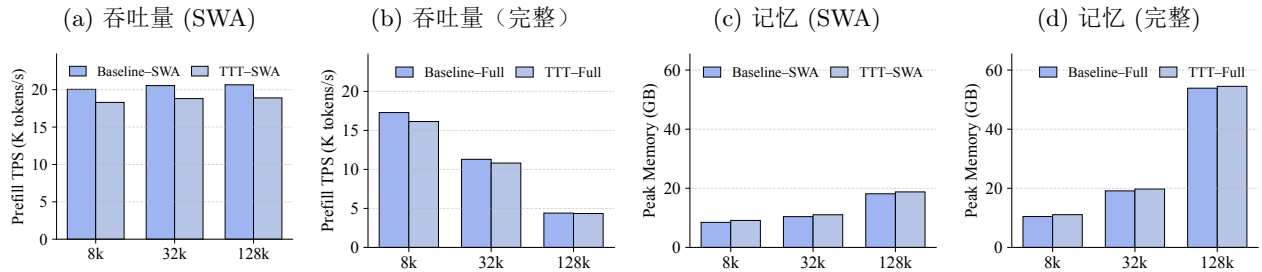


Figure 4 In-Place TTT 的效率分析。展示了使用滑动窗口注意力（SWA）和全注意力的 4B 模型在不同上下文长度下的预填充吞吐量（a, b）和峰值内存（c, d）指标。我们的 In-Place TTT 在实际场景中引入了可忽略的开销。

后被扩展到众多其他模态——包括语言 [48]、视频 [15] 和音频 [18]——凸显了其广泛的适用性。该领域的研究主要集中在两个方向以提高 TTT 的有效性：设计更复杂的测试时优化器 [?] 以及制定新颖的自监督在线学习目标 [3, 32]。然而，由于 TTT 本质上需要逐 token 的顺序更新过程，其计算效率一直是一个关键瓶颈。分块式测试时训练（chunk wise Test-Time Training）框架通过引入分块更新机制来更好地利用并行硬件，首次直接解决了这一挑战 [3, 30, 36, 49, 63, 67]。尽管取得了这些进展，但 TTT 作为主要 token 混合器的功能，需要依赖小块（small chunks）才能保持性能——而这又造成了瓶颈，限制了充分利用现代加速器所需的大规模并行性。此外，先前的工作既没有解决如何将 TTT 无缝集成到大型预训练模型中，也没有为 LLM 的自回归特性专门定制学习目标，我们的工作直接填补了这些空白。

高效长上下文架构。 另一条研究路线试图通过缓解标准注意力机制的二次复杂度来扩展 LLM 的有效上下文窗口。主要方法包括：（1）稀疏注意力方法，通过固定模式（如滑动窗口或跨步窗口）限制 token 间交互的范围 [6, 10, 65]；（2）线性时间变体，近似注意力机制或用高效的循环或门控形式替换它，例如线性注意力 [33, 43]、门控线性注意力（GLA） [58]；以及（3）状态空间模型（SSMs），将序列历史压缩为紧凑的潜在状态，从而以线性复杂度进行处理 [16, 17]。最近，*delta* 规则已成为线性注意力和 SSM 的一种流行设计选择，能够提供更好的表达性和高度可并行化的实现 [61]。这些架构上的进步与我们的框架互补。它们专注于高效处理长上下文，而 TTT 提供了一种在线适应上下文中信息的机制。我们的原位 TTT 也可以自然地与这些高效主干集成，因为它们也具有 MLP 块。我们将此留作未来的工作。

记忆设计与增强。 一个相关的研究领域涉及用显式记忆模块增强神经架构，以增强其推理和上下文理解能力。这些方法可以根据其功能大致区分：有些设计用于在外部记忆库中存储持久的、任务无关的知识，

而另一些则专注于捕获来自即时上下文的瞬态、数据相关信息 [25, 34, 35, 55, 64]。后者，即上下文记忆，已通过各种机制实现，包括循环状态转换、Transformer 中基于注意力的上下文聚合 [14]，以及对快速权重的快速、基于梯度的更新 [61]。测试时训练（TTT）是后一类的一个强大实例，它在概念上扩展了循环神经网络（RNN）中隐藏状态的概念。TTT 不是将上下文历史压缩成固定大小的激活向量，而是指定模型自身参数的一个子集——快速权重——作为高容量、动态的记忆 [2, 43]。这些权重在推理时动态更新，使模型能够持续内化不断变化的上下文信息，从而作为一种富有表现力的、在线演化的状态运行。

6 结论

我们介绍了原地测试时训练（In-Place Test-Time Training），一个实用的框架，它解决了TTT对LLMs的关键障碍。我们提出了有原则的设计选择，包括一个重用现有MLP块的原地机制、一个高效的分块更新规则，以及一个与语言建模一致的基于理论的目标。大量实验验证了我们的方法不仅作为预训练LLMs的强大的“即插即用”增强，而且在从零开始训练时也优于强基线。通过提供一个可扩展的即时适应解决方案，我们的工作为LLMs的更动态、持续学习的新范式迈出了有希望的一步。

References

- [1] Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, Alon Benhaim, Misha Bilenko, Johan Bjorck, Sébastien Bubeck, Martin Cai, et al. Phi-3 technical report: A highly capable language model locally on your phone, 2024. URL <https://arxiv.org/abs/2404.14219>.
- [2] Jimmy Lei Ba, Geoffrey E. Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu. Using fast weights to attend to the recent past. In *Advances in Neural Information Processing Systems*, 2016. URL <https://arxiv.org/abs/1610.06258>.
- [3] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2024.
- [4] Ali Behrouz, Meisam Razaviyayn, Peilin Zhong, and Vahab Mirrokni. It’s all connected: A journey through test-time memorization, attentional bias, retention, and online optimization, 2025. URL <https://arxiv.org/abs/2504.13173>.
- [5] Ali Behrouz, Meisam Razaviyayn, Peilin Zhong, and Vahab Mirrokni. It’s all connected: A journey through test-time memorization, attentional bias, retention, and online optimization. *arXiv preprint arXiv:2504.13173*, 2025.
- [6] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020. URL <https://arxiv.org/abs/2004.05150>.
- [7] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. Piqa: Reasoning about physical commonsense in natural language, 2019. URL <https://arxiv.org/abs/1911.11641>.
- [8] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [9] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, et al. Mle-bench: Evaluating machine learning agents on machine learning engineering. *arXiv preprint arXiv:2410.07095*, 2024.
- [10] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.
- [11] Aakanksha Chowdhery and et al. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*, 2022. URL <https://arxiv.org/abs/2204.02311>.
- [12] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv:1803.05457v1*, 2018.
- [13] OpenCompass Contributors. Opencompass: A universal evaluation platform for foundation models. <https://github.com/open-compass/opencompass>, 2023.
- [14] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988. Association for Computational Linguistics, 2019. URL <https://aclanthology.org/P19-1285/>.

- [15] Karan Dalal, Daniel Kocaja, Jiarui Xu, Yue Zhao, Shihao Han, Ka Chun Cheung, Jan Kautz, Yejin Choi, Yu Sun, and Xiaolong Wang. One-minute video generation with test-time training. In Proceedings of the Computer Vision and Pattern Recognition Conference, pages 17702–17711, 2025.
- [16] Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality, 2024. URL <https://arxiv.org/abs/2405.21060>.
- [17] Tri Dao, Albert Gu, et al. Hungry Hungry Hippos: Towards language modeling with state space models. arXiv preprint arXiv:2312.00752, 2023.
- [18] Sri Harsha Dumpala, Chandramouli Sastry, and Sageev Oore. Test-time training for speech, 2023. URL <https://arxiv.org/abs/2309.10930>.
- [19] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Brown, and et al. A mathematical framework for transformer circuits. Transformer Circuits Thread, 2021. URL <https://transformer-circuits.pub/2021/framework/index.html>.
- [20] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The pile: An 800gb dataset of diverse text for language modeling. arXiv preprint arXiv:2101.00027, 2020. URL <https://arxiv.org/abs/2101.00027>.
- [21] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. The language model evaluation harness, 07 2024. URL <https://zenodo.org/records/12608602>.
- [22] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. arXiv preprint arXiv:2012.14913, 2020.
- [23] Team GLM. Chatglm: A family of large language models from glm-130b to glm-4 all tools, 2024. URL <https://arxiv.org/abs/2406.12793>.
- [24] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. arXiv preprint arXiv:2407.21783, 2024.
- [25] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. Realm: Retrieval-augmented language model pre-training. In ICML, 2020.
- [26] Dan Hendrycks, Collin Burns, Steven Basart, Andrew Critch, Jerry Li, Dawn Song, and Jacob Steinhardt. Aligning ai with shared human values. Proceedings of the International Conference on Learning Representations (ICLR), 2021.
- [27] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In International Conference on Learning Representations, 2021. URL <https://arxiv.org/abs/2009.03300>.
- [28] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What’s the real context size of your long-context language models? arXiv preprint arXiv:2404.06654, 2024. URL <https://arxiv.org/abs/2404.06654>.
- [29] Jinwu Hu, Zhitian Zhang, Guohao Chen, Xutao Wen, Chao Shuai, Wei Luo, Bin Xiao, Yuanqing Li, and

- Mingkui Tan. Test-time learning for large language models. arXiv preprint arXiv:2505.20633, 2025. URL <https://arxiv.org/abs/2505.20633>. Accepted at ICML 2025.
- [30] Kazuki Irie and Samuel J. Gershman. Fast weight programming and linear transformers: from machine learning to neurobiology, 2025. URL <https://arxiv.org/abs/2508.08435>.
 - [31] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. Mistral 7b, 2023. URL <https://arxiv.org/abs/2310.06825>.
 - [32] Mahdi Karami and Vahab Mirrokni. Lattice: Learning to efficiently compress the memory. arXiv preprint arXiv:2504.05646, 2025.
 - [33] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and Fran  ois Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In Proceedings of the 37th International Conference on Machine Learning, Proceedings of Machine Learning Research. PMLR, 2020. URL <https://arxiv.org/abs/2006.16236>.
 - [34] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In ICLR, 2020.
 - [35] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. In NeurIPS, 2020.
 - [36] Zeman Li, Ali Behrouz, Yuan Deng, Peilin Zhong, Praneeth Kacham, Mahdi Karami, Meisam Razaviyayn, and Vahab Mirrokni. Tnt: Improving chunkwise training for test-time memorization. arXiv preprint arXiv:2511.07343, 2025.
 - [37] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. arXiv preprint arXiv:2412.19437, 2024.
 - [38] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. In-context learning and induction heads. arXiv preprint arXiv:2209.11895, 2022. URL <https://arxiv.org/abs/2209.11895>.
 - [39] OpenAI. Gpt-4 technical report. arXiv preprint arXiv:2303.08774, 2024.
 - [40] Keiran Paster, Marco Dos Santos, Zhangir Azerbayev, and Jimmy Ba. Openwebmath: An open dataset of high-quality mathematical web text, 2023.
 - [41] Leonid Pikelis, Michael Feil, Forrest Moret, Mark Huang, and Tiffany Peng. Llama 3 gradient: A series of long context models, 2024. URL <https://gradient.ai/blog/scaling-rotational-embeddings-for-long-context-language-models>.
 - [42] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. Yarn: Efficient context window extension of large language models. arXiv preprint arXiv:2309.00071, 2023.
 - [43] Imanol Schlag, Kazuki Irie, and J  rgen Schmidhuber. Linear transformers are secretly fast weight programmers. In International Conference on Machine Learning, pages 9355–9366. PMLR, 2021.
 - [44] David Silver and Richard S Sutton. Welcome to the era of experience. Google AI, 1, 2025.

- [45] Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, et al. Paperbench: Evaluating ai’s ability to replicate ai research. arXiv preprint arXiv:2504.01848, 2025.
- [46] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2023.
- [47] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In Proceedings of the 37th International Conference on Machine Learning, volume 119 of Proceedings of Machine Learning Research, pages 9229–9248. PMLR, 2020. URL <https://proceedings.mlr.press/v119/sun20b.html>.
- [48] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): Rnns with expressive hidden states. arXiv preprint arXiv:2407.04620, 2024. URL <https://arxiv.org/abs/2407.04620>.
- [49] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. arXiv preprint arXiv:2307.08621, 2023.
- [50] TogetherAI. Long data collections database, 2024.
- [51] Hugo Touvron, Théo Laver, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, and et al. Llama: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971, 2023.
- [52] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [53] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. In ICLR, 2021.
- [54] Ke Alexander Wang, Jiaxin Shi, and Emily B Fox. Test-time regression: a unifying framework for designing sequence models with associative memory. arXiv preprint arXiv:2501.12352, 2025.
- [55] Yu Wang, Yifan Gao, Xiusi Chen, Haoming Jiang, Shiyang Li, Jingfeng Yang, Qingyu Yin, Zheng Li, Xian Li, Bing Yin, Jingbo Shang, and Julian McAuley. Memoryllm: Towards self-updatable large language models, 2024. URL <https://arxiv.org/abs/2402.04624>.
- [56] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023. URL <https://arxiv.org/abs/2201.11903>.
- [57] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. arXiv preprint arXiv:2505.09388, 2025.
- [58] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. arXiv preprint arXiv:2312.06635, 2023. URL <https://arxiv.org/abs/2312.06635>.
- [59] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. arXiv preprint arXiv:2412.06464, 2024.

- [60] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. In International Conference on Machine Learning, pages 56501–56523. PMLR, 2024.
- [61] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. arXiv preprint arXiv:2406.06484, 2024. URL <https://arxiv.org/abs/2406.06484>.
- [62] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. In The Thirty-eighth Annual Conference on Neural Information Processing Systems, 2024.
- [63] Morris Yau, Sharut Gupta, Valerie Engelmayer, Kazuki Irie, Stefanie Jegelka, and Jacob Andreas. Sequential-parallel duality in prefix scannable models, 2025. URL <https://arxiv.org/abs/2506.10918>.
- [64] Hongli Yu, Tinghong Chen, Jiangtao Feng, Jiangjie Chen, Weinan Dai, Qiying Yu, Ya-Qin Zhang, Wei-Ying Ma, Jingjing Liu, Mingxuan Wang, and Hao Zhou. Memagent: Reshaping long-context llm with multi-conv rl-based memory agent, 2025. URL <https://arxiv.org/abs/2507.02259>.
- [65] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Y. X. Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention, 2025. URL <https://arxiv.org/abs/2502.11089>.
- [66] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, pages 4791–4800, 2019. URL <https://arxiv.org/abs/1905.07830>.
- [67] Tianyuan Zhang, Sai Bi, Yicong Hong, Kai Zhang, Fujun Luan, Songlin Yang, Kalyan Sunkavalli, William T. Freeman, and Hao Tan. Test-time training done right. arXiv preprint arXiv:2505.23884, 2025. URL <https://arxiv.org/abs/2505.23884>.

Appendix

A theorem 1 的证明

为完整起见，我们首先用假设推导出的精确界重述该定理。

Theorem 1 (Logit 层面的 LM 对齐目标与重建目标的影响（重述）). 在给定的设置和假设下，对于学习率 $\lambda_{lr} > 0$ ，在使用 **NTP 对齐目标** 进行一次更新步骤后，logits $\Delta \ell_n$ 的期望变化满足：

$$\text{(正确 logit 增加)} \quad \mathbb{E}[\Delta \ell_n[v^*]] \geq \lambda_{lr} \cdot c_{norm}^2 \cdot c_{align}, \quad (4)$$

$$\text{(其他 logits 几乎不变)} \quad |\mathbb{E}[\Delta \ell_n[w]]| \leq \lambda_{lr} \cdot \epsilon \cdot c_{align}, \quad \forall w \neq v^*. \quad (5)$$

相比之下，对于 **重建目标**，正确 token 的 logits 期望变化可忽略不计：

$$|\mathbb{E}[\Delta \ell_n[v^*]]| \leq \lambda_{lr} \cdot \epsilon \cdot c_{align}. \quad (6)$$

Proof. 我们从 section 3.3 定义的基本设定开始。所有先前上下文 token 对快速权重 $\mathbf{W}_{down}$ 的改变由 $\Delta \mathbf{W}_{down} = \lambda_{lr} \sum_{t \in \text{prior}} \mathbf{v}_t \mathbf{z}_t^\top$ 给出，其中我们用 λ_{lr} 表示学习率 η 以与定理陈述保持一致。查询位置 n 处任意 token w 的 logit 产生的变化为：

$$\Delta \ell_n[w] = \mathbf{e}_w^\top (\Delta \mathbf{W}_{down}) \mathbf{z}_n = \lambda_{lr} \sum_{t \in \text{prior}} \mathbf{e}_w^\top (\mathbf{v}_t \mathbf{z}_t^\top) \mathbf{z}_n. \quad (7)$$

由于 $\mathbf{e}_w^\top \mathbf{v}_t$ 和 $\mathbf{z}_t^\top \mathbf{z}_n$ 是标量，我们可以重新排列项得到：

$$\Delta \ell_n[w] = \lambda_{lr} \sum_{t \in \text{prior}} (\mathbf{e}_w^\top \mathbf{v}_t) (\mathbf{z}_t^\top \mathbf{z}_n). \quad (8)$$

为了分析期望变化，我们对表示取期望。应用期望的线性性，我们有：

$$\mathbb{E}[\Delta \ell_n[w]] = \lambda_{lr} \sum_{t \in \text{prior}} \mathbb{E}[(\mathbf{e}_w^\top \mathbf{v}_t) (\mathbf{z}_t^\top \mathbf{z}_n)]. \quad (9)$$

根据我们的设定，目标向量 $\mathbf{v}_t$ （例如 $\mathbf{e}_{x_t}$ 或 $\mathbf{e}_{x_{t+1}}$ ）被视为确定的。因此，我们可以将它们从期望中分解出来：

$$\mathbb{E}[\Delta \ell_n[w]] = \lambda_{lr} \sum_{t \in \text{prior}} \mathbf{e}_w^\top \mathbb{E}[\mathbf{v}_t \mathbf{z}_t^\top \mathbf{z}_n]. \quad (10)$$

现在，我们调用假设 2。它指出对于唯一键位置 t^* ，我们有 $\mathbb{E}[\mathbf{z}_{t^*}^\top \mathbf{z}_n] = c_{align}$ ，而对于所有其他先前位置 $t \neq t^*$ ，更新不提供信息增益，这意味着 $\mathbb{E}[\mathbf{v}_t \mathbf{z}_t^\top \mathbf{z}_n] = \mathbf{0}$ 。这将求和简化为对应位置 t^* 处键值对 (k^*, v^*) 的单个项：

$$\mathbb{E}[\Delta \ell_n[w]] = \lambda_{lr} \cdot \mathbb{E}[(\mathbf{e}_w^\top \mathbf{v}_{t^*}) \cdot (\mathbf{z}_{t^*}^\top \mathbf{z}_n)]. \quad (11)$$

我们现在分析这两种目标选择的简化表达式。

情况 1: NTP 对齐目标 ($\mathbf{v}_{t^*} = \mathbf{e}_{x_{t^*+1}}$)

首先，我们考虑正确 token $w = v^*$ 的 logit。将目标和 token 代入 [equation \(11\)](#) 得到：

$$\mathbb{E}[\Delta \ell_n[v^*]] = \lambda_{\text{lr}} \cdot \mathbb{E}[(\mathbf{e}_{v^*}^\top \mathbf{e}_{v^*}) \cdot (\mathbf{z}_{t^*}^\top \mathbf{z}_n)]. \quad (12)$$

根据 [假设 1](#)，token 嵌入具有非平凡的量级， $\|\mathbf{e}_{v^*}\|^2 \geq c_{\text{norm}}^2$ 。这给出了 [equation \(4\)](#) 中的下界：

$$\mathbb{E}[\Delta \ell_n[v^*]] \geq \lambda_{\text{lr}} \cdot c_{\text{align}} \cdot c_{\text{norm}}^2. \quad (13)$$

接下来，对于任何错误 token $w \neq v^*$ ，期望变化为 $\mathbb{E}[\Delta \ell_n[w]] = \mathbb{E}[(\mathbf{e}_w^\top \mathbf{e}_{v^*}) \cdot (\mathbf{z}_{t^*}^\top \mathbf{z}_n)]$ 。取绝对值并应用假设 1（它指出不同嵌入近似正交 ($|\mathbf{e}_w^\top \mathbf{e}_{v^*}| \leq \epsilon$)），我们得到 [equation \(5\)](#) 中的界：

$$|\mathbb{E}[\Delta \ell_n[w]]| \leq \lambda_{\text{lr}} \cdot c_{\text{align}} \cdot \epsilon. \quad (14)$$

情况 2: 重建目标 ($\mathbf{v}_{t^*} = \mathbf{e}_{x_{t^*}}$)

这里，我们分析对正确 logit $w = v^*$ 的影响。期望变化为：

$$\mathbb{E}[\Delta \ell_n[v^*]] = \mathbb{E}[(\mathbf{e}_{k^*}^\top \mathbf{e}_{v^*}) \cdot (\mathbf{z}_{t^*}^\top \mathbf{z}_n)]. \quad (15)$$

在归纳任务中，键 k^* 与值 v^* 是不同的。我们再次对这些不同 token 调用假设 1。取绝对值给出 [equation \(6\)](#) 中的界：

$$|\mathbb{E}[\Delta \ell_n[v^*]]| \leq \lambda_{\text{lr}} \cdot c_{\text{align}} \cdot \epsilon. \quad (16)$$

这确认了重建目标对正确答案 v^* 的 logit 具有可忽略的期望影响。

这两种情况的结果确立了 [theorem 1](#) 中的主张，为情境学习中 NTP 对齐目标的优越性提供了清晰的理论基础。证明完毕。 $\square$

B 上下文并行算法用于原位 TTT

为了更清晰，我们在[algorithm 1](#)中列出了我们的原位 TTT 的上下文并行实现的伪代码。

C 实验细节

本附录提供了第 4 节中结果所使用的实验设置、数据集、模型配置和训练超参数的所有细节。以下小节详细介绍了我们三组主要实验的设置：Qwen3-4B-Base 的 **持续预训练**、多个规模（500M、1.5B 和 4B）模型的 **从头预训练** 以及 **针对性消融研究**。我们的目标是提供足够的细节，以确保我们研究结果的可复现性。

Algorithm 1 使用上下文并行的就地 TTT (单层)

Require: Pre-trained weights θ (incl. $W_{\text{up}}, W_{\text{gate}}, W_{\text{down}}^{(0)}$); Conv1D kernel K ; projection W_{target} ; learning rate η .

- 1: **Input:** Sequence chunks $\{X^{(i)}\}_{i=1}^T$. ▷ Sequence partitioned for Context Parallelism (CP).
- 2: **for all** $i \in \{1, \dots, T\}$ **in parallel do** ▷ Step 1: Compute update deltas.
- 3: $H_i \leftarrow \text{AttentionBlock}(X^{(i)}; \theta)$ ▷ Standard attention, no changes required.
- 4: $U_i, G_i \leftarrow H_i W_{\text{up}}^\top, H_i W_{\text{gate}}^\top$
- 5: $Z_i \leftarrow \phi(G_i) \odot U_i$
- 6: $V_i \leftarrow \text{Conv1D}_K(X_0^{(i)}) W_{\text{target}}$ ▷ Compute NTP-aligned target with causal padding.
- 7: $\Delta W_i \leftarrow V_i^\top Z_i$ ▷ Compute gradient for the fast weight update.
- 8: **end for**
- 9: $\{S_i\}_{i=1}^T \leftarrow \text{CUMSUM}(\{\Delta W_i\}_{i=1}^T)$ ▷ Step 2: Aggregate deltas associatively.
- 10: **for all** $i \in \{1, \dots, T\}$ **in parallel do** ▷ Step 3: Apply updates and compute outputs.
- 11: $W_{\text{down}}^{(i-1)} \leftarrow W_{\text{down}}^{(0)} + \eta S_i$ ▷ Effective weight for chunk i uses updates from chunks $< i$.
- 12: $O_i \leftarrow Z_i (W_{\text{down}}^{(i-1)})^\top$
- 13: **end for**
- 14: **At document boundaries:** Reset fast weights to $W_{\text{down}}^{(0)}$.

C.1 数据集详情

对于大规模预训练、持续预训练以及消融研究，我们使用自行收集的数据集，以下给出这些数据集的详细信息。

从头预训练数据集。 预训练数据集主要包括通用英文和中文文本，以及高知识/推理密度数据、代码、数学数据和多语言文本，形成了语言多样性、知识与推理丰富内容、编程素材和数学推理的均衡混合。

持续预训练数据集。 持续预训练数据集旨在增强长上下文建模：其短文档部分遵循与预训练数据相似的分布，而长文档部分则结合了书本和仓库级代码等自然数据，以及检索增强和长上下文QA风格构建等合成数据，确保了与预训练的一致性以及对挑战性长上下文场景的覆盖。数据按两阶段训练课程组织成最大序列长度为 32k 和 128k 的子集。

C.2 训练与评估细节

训练细节。 所有模型均在 Nvidia H800 GPU 上训练，详细的训练超参数列于表 4 至 6。

评估细节。 我们采用评估框架 lm-evaluation-harness [21] 在常识推理基准上评估模型，并采用评估框架 opencompass [13] 在长上下文基准上评估模型。所有评估均在 Nvidia H800 GPU 上进行。

在对我们持续预训练的 Qwen3-4B 模型进行评估时，我们在推理时应用裁剪机制以确保快速的权重更新稳定。具体来说，如果更新 delta $\|\Delta \mathbf{W}_{\text{down}}^{(i)}\|_F$ 的 Frobenius 范数超过预定义阈值 τ ，则在应用之前将 delta 矩阵重新缩放为范数 τ ，即 $\Delta \mathbf{W}_{\text{down}}^{(i)} \leftarrow \tau \cdot \Delta \mathbf{W}_{\text{down}}^{(i)} / \|\Delta \mathbf{W}_{\text{down}}^{(i)}\|_F$ 。这可以防止累积更新随着序列长度的增加而无限地增长，从而保持长上下文推理的数值稳定性。对于 Qwen3-4B 模型的所有已报告评估，该阈值设置为 $\tau = 1\text{e-}5$ 。

为了评估我们的 In-Place TTT 的效率，我们评估了序列长度从 8k 到 128k 的预填充吞吐量和峰值内存。我们对基于 Qwen3-4B-Base 模型的持续预训练检查点运行推理。对于滑动窗口设置，我们将这些

Table 4 500M 和 1.5B 模型的训练超参数.

Hyperparameter	500M Model	1.5B Model
Optimizer	AdamW	AdamW
Learning Rate	5e-4	3e-4
Batch Size	2M tokens	4M tokens
Weight Decay	0.1	0.1
Gradient Clipping	1.0	1.0
Warmup Steps	1024	1024
Sequence Length	32,768	32,768
Tokens Trained	20B	60B
Sliding Window Size	2,048	4,096

Table 5 1.7B 模型和 4B 模型预训练的训练超参数

Hyperparameter	value
Optimizer	AdamW
Learning Rate	3e-4
Batch Size	8M tokens
Weight Decay	0.1
Gradient Clipping	1.0
Warm-up Tokens ¹	1.6B
Sequence Length	8,192
Tokens Trained	120B

Table 6 两阶段持续预训练的超参数

Hyperparameter	Stage 1 (32k Context)	Stage 2 (128k Context)
Base Model	Qwen3-4B-Base	Qwen3-4B-Base
Optimizer	AdamW	AdamW
Learning Rate	5e-6	5e-6
Weight Decay	0.1	0.1
Sequence Length	32,768	131,072
Tokens Trained	~20B	~15B
RoPE Extension	None	YaRN
Conv Size	5	5

预训练检查点的注意力机制手动更改为 1024 个 token 的滑动窗口。我们在 Nvidia H800 GPU 上以 batch size 为 1 运行推理。

C.3 模型配置细节

本节详细介绍了实验中所用模型的架构配置。所有模型均为 decoder-only Transformer 架构，采用标准组件，包括 SwiGLU 激活函数和旋转位置编码 (RoPE) [46]。从头训练的所有模型的关键架构参数总结在表 8 中。

从头训练的模型根据其实验目的采用不同的注意力机制。500M 和 1.5B 模型使用滑动窗口注意力，其

Table 7 用于 LLaMA-3.1-8B 和 Qwen3-14B-Base 持续预训练的超参数。

Hyperparameter	LLaMA-3.1-8B	Qwen3-14B-Base
Optimizer	AdamW	AdamW
Learning Rate	5e-6	5e-6
Weight Decay	0.1	0.1
Sequence Length	32,768	32,768
Tokens Trained	~20B	~20B
RoPE Extension	None	None
Conv Size	5	5

Table 8 500M 和 1.5B 模型的架构配置。

Parameter	500M	1.5B
Parameters (Approx.)	500M	1.5B
Hidden Size (d_{model})	1024	2048
Num Layers	24	24
Num Attention Heads	8	16
FFN Hidden Size (d_{ff})	3072	6144
Window Size	2048	4096
Vocabulary Size	32,000	32,000
Rope Base	1e6	1e6

模型配置列在 table 8 中。4B 规模的实验和消融研究评估了两种变体：一种使用全注意力，另一种使用滑动窗口注意力。4B 模型和 1.7B 模型的骨干架构与 Qwen3-4B-Base 模型和 Qwen3-1.7B-Base 模型相同。

对于 section 4.1 中描述的持续预训练实验，我们直接从公开可用的预训练模型开始——Qwen3-4B-Base [57]、LLaMA-3.1-8B [24] 和 Qwen3-14B-Base [57]——继承其架构而不进行修改。在采用我们方法的实验中，In-Place TTT 模块被集成到 MLP 块中，并每六层应用一次。对于消融研究，该频率如主论文所述进行变化。Qwen3-4B-Base 的训练超参数列于 table 6 中，而 LLaMA-3.1-8B 和 Qwen3-14B-Base 的超参数列于 table 7 中。

In-Place TTT 模块的初始化。 当将 In-Place TTT 集成到预训练模型中进行持续训练时，仔细的初始化对于在训练开始时保持模型的预训练能力至关重要。具体来说，我们初始化新引入的 TTT 组件——Conv1D 算子和投影矩阵 $\mathbf{W}_{\text{target}}$ ——使得 TTT 更新 $\Delta \mathbf{W}_{\text{down}}$ 在初始化时可忽略，从而确保模型从其原始预训练行为开始。具体而言，深度可分离 Conv1D（核大小 5，因果填充，无偏置）被零初始化，因此目标 $\hat{\mathbf{V}}$ 在初始化时为零。投影矩阵 $\mathbf{W}_{\text{target}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ 被初始化为稀疏对角矩阵，其中所有非对角元素为零，对角元素从 $\mathcal{N}(0, \sigma^2)$ 中采样，其中 σ 是模型的标准初始化器范围。这种两个组件接近零的初始化保证了初始快速权重更新 $\eta \hat{\mathbf{V}}_{[i]}^T \mathbf{Z}_{[i]} \approx \mathbf{0}$ ，因此有效的 $\mathbf{W}_{\text{down}}$ 保持与其预训练值相同。随着训练的进行，Conv1D 和投影逐渐学习产生有意义的 NTP 对齐目标，使 TTT 机制平稳出现，而不会破坏预训练模型。

D LLMs 的使用

在准备本手稿期间，使用了 LLMs 来检查语法并提高可读性。所有作者均已审阅、编辑并对论文的最终版本承担全部责任。